ANYBOT PLATFORM | BRAINSWARM ROBOTICS

Practical Session 2

Topics, services, actions, and commanding a real manipulator

Audience	Undergraduate students, second practical session
Duration	Approximately 2 hours
Robot	Yahboom DofBot manipulator. One 15 minute live slot per student.
Package	dofbot_practice, supplied in src/Practical_Session_2, in C++ and Python
Outcome	A C++ subscriber reading joint states and a publisher commanding the arm

1. What this session is for

In the first session everything you wrote ran in isolation. Nothing you typed reached a robot. This session closes that gap. You will learn how ROS 2 nodes talk to one another, then use that knowledge to read the state of a real manipulator arm and to command it, from code you have written and built yourself.

The session is organised so that every student gets hands on time with the physical arm. You will write and test your code in your own preview workspace, which is always available, and then book a short live slot to run it against the hardware.

2. Before you arrive

This session assumes the first one. If any of the following is uncertain, read the Practical Session 1 handout again and work through it, because we will not be repeating it.

1. Creating a workspace and building it with colcon.
2. Sourcing the ROS 2 installation and your own workspace in every new terminal.
3. Creating a package and understanding what package.xml and CMakeLists.txt control.
4. Adding an install block so that ros2 run can find your program.

> **One difference from last time** Session 1 used Python with install(PROGRAMS ...). This session uses C++, which compiles, so the install block uses install(TARGETS ...) instead. The starter package already has this written for you.

3. How the two hours are organised

The teaching runs continuously. Live robot slots run alongside it, so while one student is on the arm the rest of the group is writing and building code. Your slot is short on purpose: it is enough to run what you prepared, and it keeps the arm moving through the whole group.

Time	Activity
0:00 to 0:25	Topics and message types. Inspecting a running system from the terminal.
0:25 to 0:45	Services and actions. When each one is the right choice.
0:45 to 1:05	Publishers and subscribers as code. Walkthrough of the starter package.
1:05 to 1:40	Exercise A, the joint state subscriber. Build, run, modify.
1:40 to 2:00	Exercise B, commanding the arm. Live slots continue to the end.

> **Robot slots** Each student books one 15 minute slot on the manipulator. The platform enforces this length, so a longer booking will be refused. Book early in the session, pick a time at least 20 minutes ahead, and use the waiting time to get your code building cleanly in your preview workspace.

4. Topics and message types

A topic is a named channel. Any node may publish messages onto it, and any number of nodes may subscribe to it. Publishers and subscribers do not know about each other, and neither waits for the other. This is the right tool for a continuous stream of data, such as the position of each joint reported many times a second.

Inspecting a live system

With the robot running, these commands tell you what exists without writing any code:

```
ros2 topic list # every topic currently advertised
ros2 topic info /joint_states # its type, and how many publishers and subscribers
ros2 topic echo /joint_states # print the messages as they arrive
ros2 topic hz /joint_states # how many messages per second
```

Message types

Every topic carries exactly one type, and both sides must agree on it. A subscriber expecting one type will simply never receive from a publisher sending another. To see the structure of a type:

```
ros2 interface show sensor_msgs/msg/JointState
```

For the arm, the type that matters most is sensor_msgs/msg/JointState. Its useful fields are parallel arrays, which is to say that name[0] describes position[0], name[1] describes position[1], and so on.

Field	Meaning
header	Timestamp and frame. Use it to tell stale data from fresh data.
name	The names of the joints, in order.
position	The angle of each joint in radians, in the same order as name.
velocity	Rate of change for each joint. May be empty if the driver does not report it.
effort	Force or torque per joint. Also may be empty.

> **Do not assume the arrays are full** A publisher is allowed to send names with no velocities. Code that indexes position[i] without checking the length will crash on the real robot even though it worked in simulation. The starter code shows the guard.

5. Services and actions

Topics are the wrong tool when you want one specific thing done and an answer about whether it worked. ROS 2 offers two further mechanisms for that.

Mechanism	Shape	Use it when
Topic	One way, continuous, no reply	Data flows constantly and nobody needs an acknowledgement. Joint states, camera frames.
Service	Request and reply, immediate	You want one short job done and a result straight back. Reading a parameter, resetting a counter.
Action	Goal, feedback, result, cancellable	The job takes time, you want progress while it runs, and you may need to stop it. Moving an arm to a pose.

Trying them from the terminal

```
ros2 service list
ros2 service type /some_service
ros2 service call /some_service std_srvs/srv/Trigger


ros2 action list
ros2 action info /arm_controller/follow_joint_trajectory
ros2 action send_goal <action_name> <action_type> "{ ... }"
```

Moving a physical arm is the textbook case for an action. The motion takes a few seconds, you want to know how far along it is, and you need the ability to cancel it if something looks wrong. That last point is not theoretical when the hardware is on the desk in front of you.

6. Publishers and subscribers in code

A subscriber asks for a topic by name and type, and hands over a function to be called each time a message arrives. A publisher asks for a topic by name and type, and then sends whenever something tells it to. Three points are worth fixing in your mind now, because they account for most of the confusion later.

- **The callback runs on someone else's schedule.** You do not call it. Never block inside it, because nothing else can be delivered until it returns.
- **A publisher on its own sends nothing.** Something has to call publish. In the starter code a timer does this twice a second.
- **spin is what keeps a node alive.** Without it the node is constructed and then the program immediately ends, having heard nothing.

7. The starter package

A package named dofbot_practice is already in your workspace under src/Practical_Session_2. It builds and runs as given, which lets you confirm your toolchain before you change anything. It holds two exercises, each supplied in both C++ and Python. Use whichever language you prefer. Reading both is worthwhile, because the concepts are identical and only the syntax differs.

Exercise	C++	Python
A. Subscribe to /joint_states and print what the arm reports.	src/joint_state_subscriber.cpp	scripts/joint_state_subscriber.py
B. Publish a joint trajectory on a timer. Holds every joint at zero until you change it.	src/arm_command_publisher.cpp	scripts/arm_command_publisher.py

The Python files keep their .py extension when installed, so they never collide with the compiled nodes and you can run both at once. Note how CMakeLists.txt installs each kind, because the distinction is the whole reason ros2 run can find them:

Command	For	You name
install(TARGETS ...)	Programs CMake compiled	The target, because CMake already knows where the binary went.
install(PROGRAMS ...)	Files already runnable as they are	The path, because CMake only copies them. This is the Session 1 block, unchanged.

> **Two requirements for the Python nodes** They must be executable in the source tree (chmod +x) and must start with a #!/usr/bin/env python3 line. Miss either and they install without error but will not run.

Build it, source the workspace, and confirm both executables exist before going further:

```
cd ~/dofbot_ws
colcon build --packages-select dofbot_practice
source install/setup.bash
ros2 pkg executables dofbot_practice
```

The last command should list all four executables. If it lists nothing, or lists the compiled nodes but not the Python ones, the install blocks in CMakeLists.txt are the place to look, exactly as in the first session.

8. Exercise A, reading the arm

Run the subscriber while the arm is powered and reporting. Move the arm by hand if it is safe to do so, or run Exercise B alongside it, and watch the numbers change.

```
ros2 run dofbot_practice joint_state_subscriber # C++
ros2 run dofbot_practice joint_state_subscriber.py # Python
```

Then make these three changes, in order. Each is marked with a TODO comment in the file.

1. Print only one joint instead of all of them. Pick a name from the output you just saw and skip every other name in the callback.
2. Warn when a joint leaves a range you choose. Use RCLCPP_WARN rather than RCLCPP_INFO so the message stands out in the log.
3. Change the topic name to something that does not exist. The node will start normally and print nothing at all. Understand why before changing it back.

> **The third change is the point of the exercise** A subscriber that hears nothing looks identical to a robot that is not publishing. Checking with ros2 topic list and ros2 topic info, rather than guessing, is the habit this session is trying to build.

9. Exercise B, commanding the arm

This is the part that moves real hardware. Read the safety note before you run anything.

> **Safety** Keep your first movements small, well under half a radian from the current pose. Watch the arm itself rather than the screen the first time you run a new target. Keep a hand near the power switch. If the arm does something you did not intend, stop the node with Ctrl and C, and cut power if it does not settle.

First confirm the topic the controller is actually listening on, because this differs between configurations and a trajectory sent to the wrong name goes nowhere:

```
ros2 topic list | grep -i trajectory
ros2 topic info /arm_controller/joint_trajectory
```

Run the publisher as given. It holds every joint at zero, so nothing should move. Confirm that with your subscriber from Exercise A running in a second terminal:

```
ros2 run dofbot_practice arm_command_publisher # C++
ros2 run dofbot_practice arm_command_publisher.py # Python
```

Then work through the TODO comments in the file:

1. Put a small non zero angle on one joint and confirm that joint, and only that joint, moves.
2. Change time_from_start and observe the arm move more slowly or more quickly to the same pose.
3. Alternate between two poses so the arm sweeps back and forth under your control.

Doing the same thing without writing code

Anything your node can publish, the command line can publish too. This is invaluable for deciding whether a problem is in your code or in the robot:

```
ros2 topic pub --once /arm_controller/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint1'], points: [{positions: [0.2], time_from_start: {sec: 2}}]}"
```

If the command line moves the arm and your node does not, the fault is in your node. If neither moves it, the fault is in the topic name, the controller, or the robot itself. That single comparison will save you more time than any other technique in this session.

10. If something does not work

Symptom	Most likely cause
ros2 run reports no executable found	The install block in CMakeLists.txt does not list the node, or the workspace has not been sourced since the last build.
The node starts but prints nothing	Wrong topic name, or nothing is publishing. Check with ros2 topic list and ros2 topic info.
Command not found for ros2	The ROS 2 installation has not been sourced in this terminal.
Build succeeds but changes have no effect	The workspace was not sourced again, so an older installed copy is still being run.
The arm does not move and there is no error	The trajectory went to a topic nobody is listening on. Compare against ros2 topic pub.

11. What you should be able to do afterwards

1. Explain the difference between a topic, a service and an action, and choose correctly between them for a given job.
2. Inspect an unfamiliar running system from the terminal and work out what it publishes.
3. Read the definition of a message type and write code against it safely.
4. Write, build and run a subscriber in C++ that reacts to live robot data.
5. Write, build and run a publisher that commands a physical manipulator.
6. Decide whether a fault is in your code or in the robot, using evidence rather than guesswork.